

Test Fundamentos Unidad 1

PREGUNTAS

Pregunta 1

Cuando hablamos de "paradigma de programación", ¿a qué nos estamos refiriendo realmente?

- A)** A la versión del lenguaje que estamos usando
- B)** A la forma de pensar y estructurar soluciones en código, incluyendo cómo modelamos el estado y razonamos sobre la corrección
- C)** A la cantidad de librerías que tiene un lenguaje
- D)** A las convenciones de estilo (sangrías, nombres de variables, etc.)

Pregunta 2

¿Cuál de las siguientes descripciones encaja mejor con el paradigma imperativo?

- A)** Describes el resultado que quieres y el motor decide cómo obtenerlo
- B)** Defines reglas y hechos, y un motor de inferencia deduce las respuestas
- C)** Te centras en una secuencia de instrucciones que modifican un estado mediante variables y estructuras
- D)** Solo utilizas funciones matemáticas sin efectos secundarios

Pregunta 3

¿Por qué se dice que lenguajes como SQL se consideran declarativos?

- A)** Porque obligan a declarar todas las variables al principio del programa
- B)** Porque describen "qué" se quiere obtener, dejando que el motor decida el "cómo" ejecutarlo

- C) Porque no permiten trabajar con bases de datos reales
- D) Porque siempre son más rápidos que cualquier enfoque imperativo

Pregunta 4:

¿Cuál de las siguientes características es típica del enfoque funcional?

- A) Uso intensivo de variables globales modificables
- B) Funciones puras, sin efectos secundarios, y preferencia por estructuras inmutables
- C) Dependencia total de bucles for y while para cualquier operación
- D) Necesidad de programar siempre en lenguajes de bajo nivel

Pregunta 5

En un backend moderno, ¿cuál de estas afirmaciones refleja mejor el uso real de los paradigmas?

- A) Se elige un único paradigma y se evita mezclar estilos para no "contaminar" el código
- B) Se combinan paradigmas (imperativo, declarativo, funcional, lógico) según la capa y el problema a resolver
- C) Solo se permite el paradigma funcional por motivos de rendimiento
- D) El paradigma lógico se usa para absolutamente toda la aplicación

Pregunta 6

¿Para qué se utiliza principalmente la notación Big-O en análisis de algoritmos?

- A) Para calcular el tiempo exacto en segundos de ejecución de un programa
- B) Para estimar el consumo energético de un algoritmo
- C) Para clasificar cómo crece el uso de tiempo o espacio de un algoritmo en función del tamaño de la entrada
- D) Para contar cuántas líneas de código tiene un proyecto

Pregunta 7

¿Cuál de las siguientes situaciones se corresponde mejor con una complejidad $O(n)$?

- A)** Acceder a un elemento concreto de un array por índice
- B)** Recorrer todos los elementos de una lista con un único bucle
- C)** Realizar una búsqueda binaria en una lista ordenada
- D)** Un algoritmo que compara cada elemento con todos los demás (doble bucle anidado)

Pregunta 8:

¿Qué idea resume mejor el enfoque de diseño incremental con MVP (Producto Mínimo Viable)?

- A)** Diseñar toda la aplicación completa en detalle antes de escribir una sola línea de código
- B)** Construir una versión mínima pero funcional, probarla, recibir feedback y ampliarla paso a paso sin romper lo anterior
- C)** Publicar directamente la versión más compleja del producto para ahorrar iteraciones
- D)** Trabajar sin plan ni prioridades, añadiendo funcionalidades según se te ocurran

Pregunta 9:

Cuando hablamos de "refactorizar" código, ¿qué se pretende conseguir?

- A)** Cambiar la funcionalidad del programa para añadir nuevas características
- B)** Mejorar la estructura interna del código sin modificar su comportamiento externo
- C)** Reducir el número de archivos del proyecto a uno solo
- D)** Eliminar todos los comentarios para que el código sea más "limpio"

Pregunta 10:

¿Qué principio de Clean Code se está aplicando cuando renombramos variables genéricas como x o aux a nombres más expresivos como totalVentas o saldoCuenta?

- A)** Evitar duplicación
- B)** Mantener funciones cortas
- C)** Usar nombres significativos que expliquen su propósito
- D)** Eliminar la necesidad de comentarios en el código por completo

HOJA DE RESPUESTAS

Pregunta	Respuesta	Justificación
1	B	Un paradigma es una forma de pensar y estructurar el código, no solo una cuestión de sintaxis o versión del lenguaje
2	C	El imperativo se basa en dar pasos que modifican un estado mediante variables y estructuras
3	B	En el paradigma declarativo se indica el resultado deseado y el motor decide el plan de ejecución
4	B	El enfoque funcional prioriza funciones puras e inmutabilidad, reduciendo efectos secundarios
5	B	En la práctica se mezclan paradigmas según la capa: datos, lógica, reglas, orquestación, etc.
6	C	Big-O clasifica el crecimiento de tiempo o espacio respecto al tamaño de la entrada (n)
7	B	Recorrer una lista completa con un bucle

Eliminado: <object>

PROMETEO

Pregunta	Respuesta	Justificación
8	B	simple es el ejemplo típico de complejidad lineal $O(n)$ El diseño incremental con MVP busca tener siempre una versión funcional que se mejora con feedback
9	B	Refactorizar es mejorar la estructura interna manteniendo la misma funcionalidad observable
10	C	Clean Code propone nombres significativos para que el código se entienda casi sin esfuerzo extra
.		